Year: 2026-2027 Sem V-NEP

Course: Agile Software Development & DevOps Lab

Class: T1 Date of Submission: 05-09-2026

A Mini Project Report on

QuizKaro

An Adaptive Quiz Platform Delivered Through an Agile and DevOps Pipeline

Submitted in partial fulfilment of the requirements of the course

Agile Software Development & DevOps Lab

Semester V (NEP) · Academic Year 2026–2027 · Class T1

Submitted by

Name	Roll No.
Ryaan Devnani	**2401023**
Preet Dayaramani	**2401021**
Dinky Aswani	**2401006**

Under the guidance of

Prof. Sanober Shaikh

Department of Artificial Intelligence & Data Science

Thadomal Shahani Engineering College

2026–2027

CERTIFICATE

This is to certify that the mini project entitled “QuizKaro: An Adaptive Quiz Platform Delivered Through an Agile and DevOps Pipeline” is a bona fide work of Ryaan Devnani (2401023), Preet Dayaramani (2401021) and Dinky Aswani (2401006), submitted in partial fulfilment of the requirements of the course Agile Software Development & DevOps Lab, Semester V, during the academic year 2026–2027.

Prof. Sanober Shaikh — Project Guide

Madhuri Rao — Head of Department

Date: 05-09-2026 Place: TSEC, Bandra, Mumbai

ABSTRACT

Most quiz applications show a question, count the score and stop. Every learner receives the same questions, answers are not explained, weak topics stay hidden, and a teacher cannot see how a group is doing. QuizKaro addresses this with a web-based adaptive quiz platform. Learners take timed quizzes whose difficulty reacts to their answers and then see an instant score with explanations, a leaderboard and a report of the topics they keep getting wrong. Administrators create quizzes with five question types, form groups, assign quizzes, host live group quizzes and download learner and organisation reports as PDF, CSV and Excel files.

The application was used as the vehicle for applying the seven experiments of the Agile Software Development & DevOps Lab to a single product. Work was planned in Jira as a Scrum sprint of six issues (QUIZ-1 to QUIZ-6), with automation rules that move an issue to In Progress, In Review and Done when its branch is created, its pull request is opened and the pull request is merged. Each change is developed on a branch named after its issue and merged into main through a pull request that carries a Definition of Done checklist; Netlify builds a deploy preview for every pull request and deploys main to the public site. The front end is containerised with a multi-stage Docker build (Node builds, nginx serves) and composed with an Express API, MySQL 8.4, phpMyAdmin, Prometheus and Grafana into a six-service Docker Compose stack. A Jenkins pipeline with Build, Test and Deploy stages was set up and run against a GitHub repository, and Prometheus and Grafana monitor the availability of the API.

The delivered system comprises 28 quizzes with 168 questions, a TypeScript front end covered by 20 automated tests with type-checking, linting and the production build all passing, a container image of about 95 MB that uses under 9 MB of memory when idle, and a Compose stack in which all services were running about twelve seconds after start-up. An end-to-end check stored a quiz attempt through the API in MySQL and displayed it in phpMyAdmin. The report also records the limits of the work honestly: authentication and role checks are client-side demonstration features, and some parts of the pipeline were validated on a demonstration repository rather than on the QuizKaro repository itself.

Keywords: Agile, Scrum, Jira, Git, GitHub, Docker, Docker Compose, Jenkins, CI/CD, Netlify, Prometheus, Grafana, TypeScript, Vite

CONTENTS

Chapter	Title
1	Introduction
2	Requirements and Agile Planning (Experiment 6)
3	System Design
4	Application Implementation
5	DevOps Implementation (Experiments 1–7)
6	Testing
7	Results and Discussion
8	Conclusion and Future Scope
	References
A	Appendix: Running the Project

1. Introduction

1.1 Background

Online quizzes are widely used for practice and revision, but a simple implementation shows the same questions to every learner, gives only a final score, and offers the teacher no view of how a class is doing. A learner who keeps missing questions on one topic is not told which topic it is, and an organisation that wants to keep a record has to copy results by hand.

The Agile Software Development & DevOps Lab introduced, through seven separate experiments, the practices used to deliver software reliably: version control, containers, multi-container orchestration, continuous integration, automated deployment, Agile project tracking and monitoring. Each experiment was performed in isolation on a toy workload. This mini project applies all seven to one realistic product so that their interaction, rather than each tool on its own, becomes visible.

1.2 Problem Statement

To design, build and operate a web application that adapts quiz difficulty to each learner, explains answers, reports weak topics and gives administrators groups, assignments and downloadable reports, and to deliver it through an Agile, largely automated pipeline in which every change is planned as a Jira issue, versioned in Git, checked automatically, deployed as a website and as containers, and observed by monitoring tools.

1.3 Objectives

1. Allow learners to sign in, take timed quizzes with five question types, see instant feedback with explanations, a leaderboard and a star rating.
2. Adapt the difficulty of the next question to the learner's answers, award XP and levels, and recommend the next quiz and quizzes that match chosen skills.
3. Give administrators a dashboard of learners' statistics, quiz creation, groups, assignments, configuration and live group quizzes, and let both roles download reports as PDF, CSV and Excel.
4. Plan the work as a Scrum sprint in Jira with a Definition of Done linked to pull requests and automated checks.
5. Version every change on a branch named after its Jira issue and merge only through pull requests.
6. Containerise the application and run the complete stack (web, API, database, database admin, Prometheus and Grafana) with one Docker Compose command.
7. Run an automated build, test and deploy pipeline, and deploy main to a public URL automatically with a preview for every pull request.
8. Monitor the API over time and raise an alert when it is down.

1.4 Scope

QuizKaro runs in two modes that share the same front-end code. In standalone mode, which Netlify serves, the browser keeps all data in localStorage and no server code runs. In full-stack mode, started with Docker Compose, an Express API stores quiz attempts in MySQL so that the leaderboard is shared. Sign-in and role checks are performed in the browser and are intended for demonstration only. Live group quizzes link tabs of one browser. Payment, e-mail notification and AI-generated questions are out of scope. The deployment targets are Netlify for the public site and a single Docker host for the full stack.

1.5 Mapping of Lab Experiments to the Project

Exp	Lab topic	Application in QuizKaro	Section
1	Version control with Git	Branch per Jira issue (QUIZ-n-slug), pull request with template, merge into main, Netlify deploy preview	5.1
2	Containerisation with Docker	Multi-stage Dockerfile: Node builds the site, nginx serves it; container run, logged, inspected and restarted	5.2
3	Docker Compose	Six-service stack: web, API, MySQL, phpMyAdmin, Prometheus, Grafana; health-gated database, named volumes	5.3
4	Jenkins CI pipeline	Declarative pipeline with Build, Test and Deploy stages run against a GitHub repository	5.4
5	Automated deployment	Netlify builds and deploys main on every push and previews every pull request; Jenkins Deploy stage	5.5
6	Agile project lifecycle	Jira Scrum project QUIZ, sprint of six issues, board, automation rules linked to GitHub	2
7	Monitoring	Prometheus scrapes the API's /metrics endpoint; Grafana dashboard "QuizKaro Monitoring"; alert when the API is down	5.6

Table 1.1: Lab experiments and where each is applied

2. Requirements and Agile Planning

2.1 User Roles

Role	Goal	Main screens
Learner	Practise at the right level, understand mistakes and find weak topics	Home, quiz, result, weak topics, skills, live quiz, reports
Administrator	Create and assign quizzes, watch learners' progress and download reports	Dashboard, create quiz, groups, configuration, live host, reports

Table 2.1: User roles

2.2 Functional Requirements

- Users sign in with an account; the role (learner or admin) decides which pages and tools appear. Nine demonstration accounts are provided.
- The home page lists quizzes by category, shows only two rows with a "See all" button, and recommends the next quiz and quizzes matching the learner's skills.
- A quiz shows one question per screen with a timer, colour feedback and an explanation, and supports multiple choice, match the following, fill in the blanks, drag to order and free text, with optional images.
- Difficulty adapts: two correct answers in a row raise the level, a wrong answer lowers it. XP is awarded by difficulty and decides the learner's level.
- When time runs out the quiz is submitted automatically with the answers given so far.
- The result screen shows score, XP, level, topic accuracy, pass or fail, an answer review, a top-five leaderboard and a star rating; "Retry my mistakes" replays only the wrong questions.
- Administrators create quizzes with all question types, form groups by importing learners, assign quizzes, configure rules (time, word limit, pass mark, attempts) and host live quizzes with a room code.
- Learners and administrators can download reports as PDF, CSV and Excel; administrators can do so for the whole organisation or any learner.

2.3 Non-Functional Requirements

- Performance: the application is a static bundle with no server round trip needed to take a quiz; the timer ticks every 250 ms.
- Security: all user-typed text is HTML-escaped before display; the API validates input and uses parameterised SQL. Passwords and role checks are not production-grade (see Section 7.4).
- Reliability: the database and web containers declare health checks; the front end falls back silently to local data if the API does not answer; quizzes auto-submit when time ends.
- Observability: the API exposes request and saved-attempt counters at /metrics; Prometheus scrapes them and an alert fires if the API is down.
- Portability: the complete stack starts with one docker compose up --build command on any machine with Docker.

2.4 Scrum Setup

The project was organised with Scrum, following the Agile project lifecycle of Experiment 6. A Jira software space named QuizKaro (key QUIZ) holds the backlog and board. Application features were first planned as thirteen stories (QZ-1 to QZ-13, 54 story points) over two sprints, documented in the repository file docs/AGILE.md. The DevOps work of this lab was then planned as QUIZ Sprint 1, a one-week sprint from 13 to 20 October 2026 with the goal “Containerise, automate and monitor QuizKaro” (Figure 2.1). The six issues were created in the backlog (Figure 2.2) and moved onto the board, where all of them start in To Do (Figure 2.3).

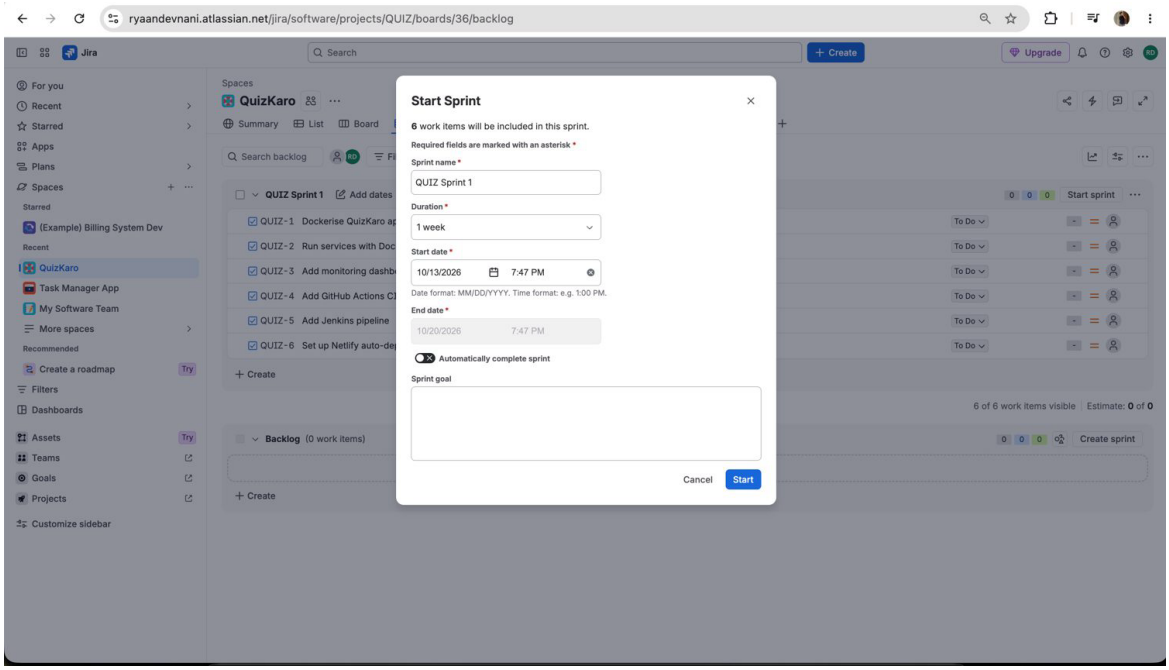


Figure 2.1: Starting QUIZ Sprint 1 with six work items and a one-week duration

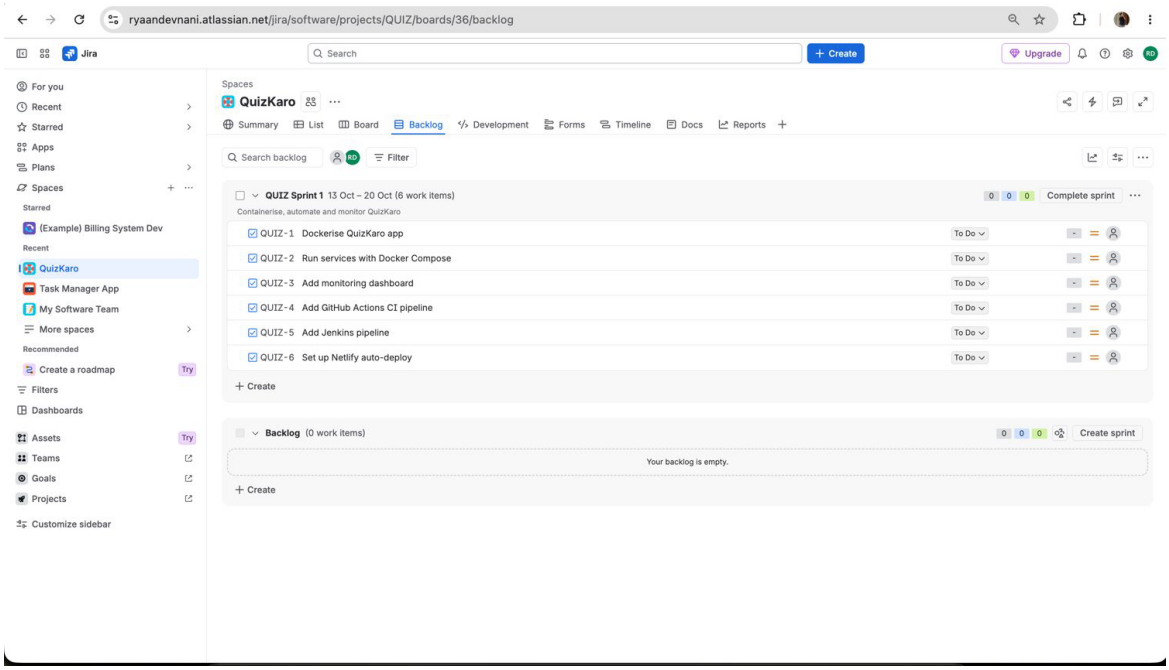


Figure 2.2: Sprint backlog showing issues QUIZ-1 to QUIZ-6

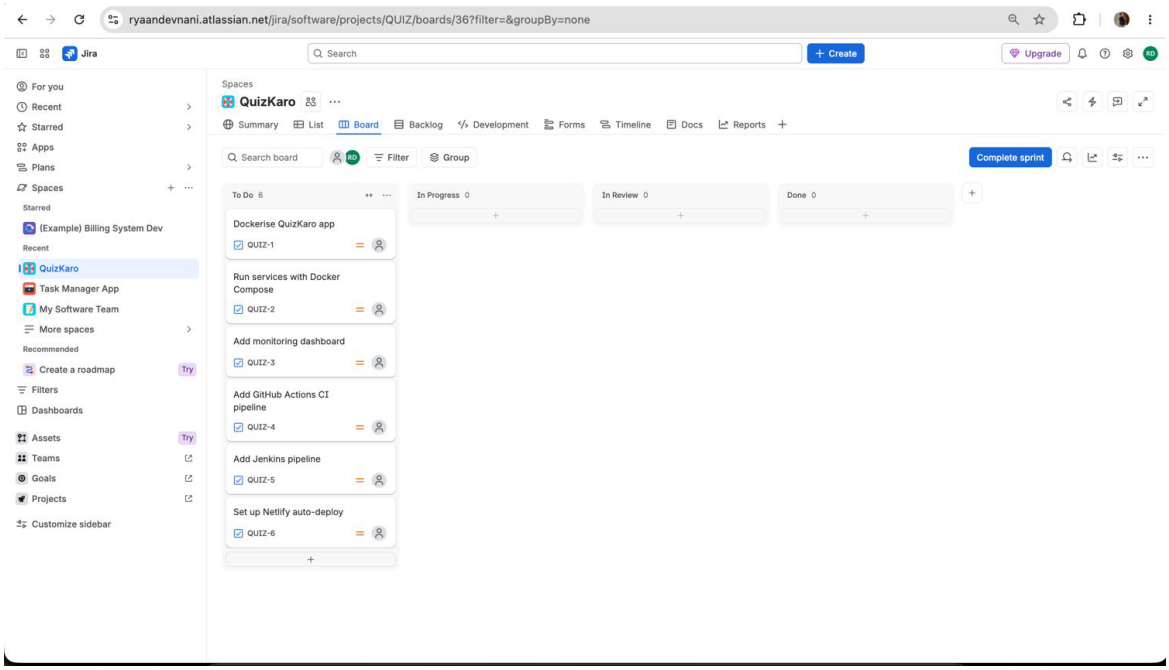


Figure 2.3: Scrum board at the start of the sprint, all issues in To Do

The issue key is written into the branch name, the commit message and the pull-request title, so any change can be traced back to the requirement it delivers. To keep the board honest without manual dragging, three Jira automation rules were enabled and linked to GitHub (Table 2.5, Figure 2.4). After the first branch was pushed, QUIZ-1 moved to In Progress by itself (Figure 2.5).

Rule	Trigger	Action
Branch created → In Progress	A branch containing the issue key is created in GitHub	Transition the work item to In Progress
PR created → In Review	A pull request is opened	Transition the work item to In Review
PR merged → Done	The pull request is merged	Transition the work item to Done

Table 2.5: Jira automation rules

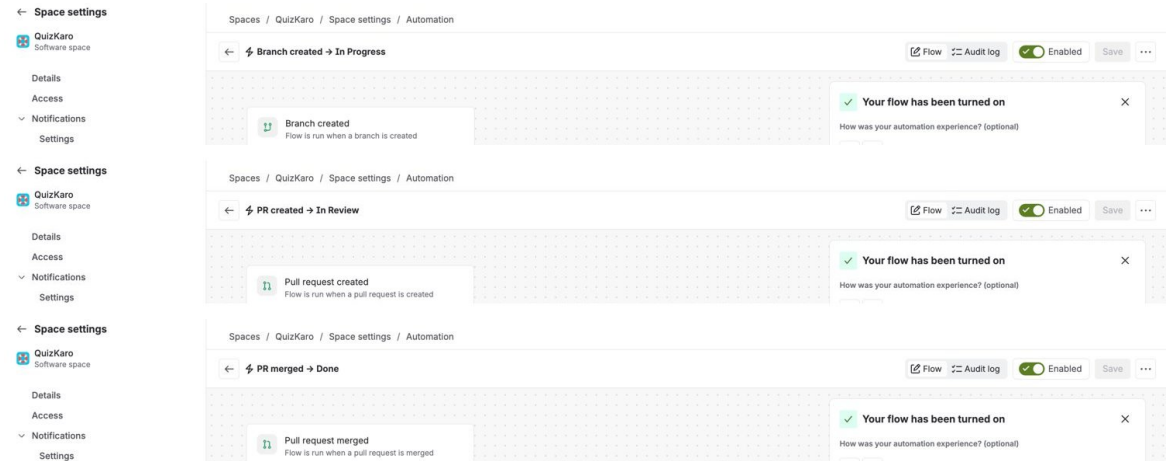


Figure 2.4: The three Jira automation rules, all enabled

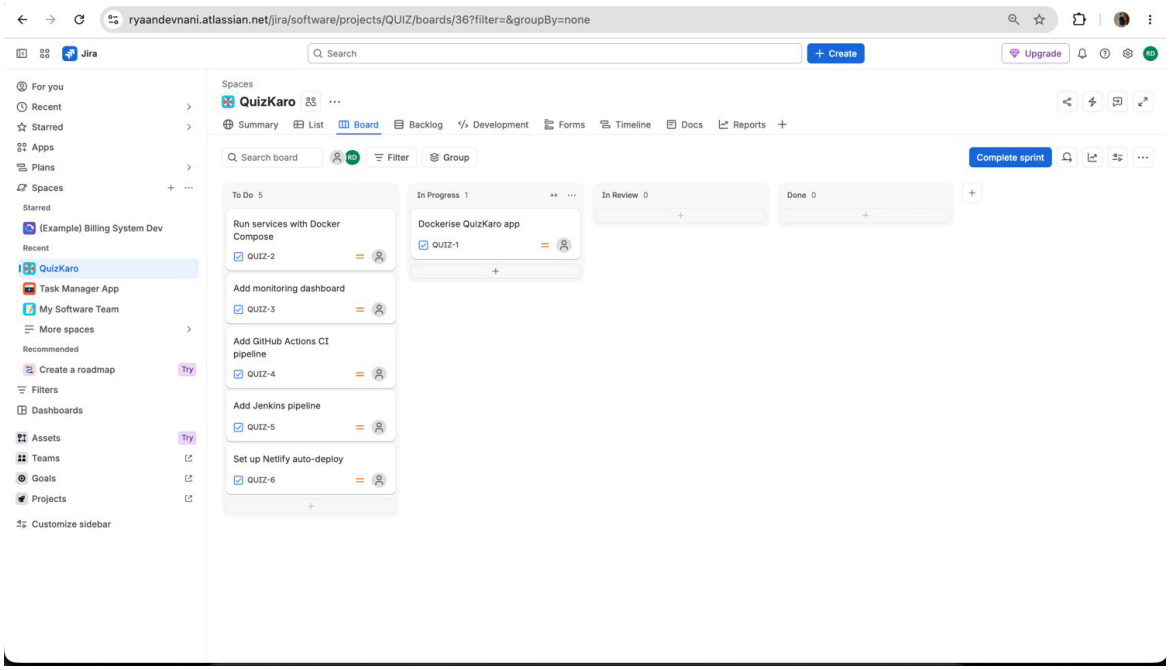


Figure 2.5: QUIZ-1 moved to In Progress when its branch was created

2.5 Product Backlog

Key	Type	Summary	Owner	Sprint
QUIZ-1	Task	Dockerise QuizKaro app	Ryaan Devnani	1
QUIZ-2	Task	Run services with Docker Compose	Dinky Aswani	1
QUIZ-3	Task	Add monitoring dashboard	Dinky Aswani	1
QUIZ-4	Task	Add GitHub Actions CI pipeline	Preet Dayaramani	1
QUIZ-5	Task	Add Jenkins pipeline	Ryaan Devnani	1
QUIZ-6	Task	Set up Netlify auto-deploy	Preet Dayaramani	1

Table 2.2: Product backlog of project QUIZ (DevOps sprint)

These items are technical tasks with no user-facing behaviour, so, as in Experiment 6, they carry no story points. The application features themselves (adaptive quiz, five question types, reports, groups and live quiz) were delivered earlier as stories QZ-1 to QZ-13.

2.6 Sprint Plan

Sprint	Goal	Scope
Application sprints 1–2	Core quiz first, then smart features, admin tools and reports	Stories QZ-1 to QZ-13, 54 story points (docs/AGILE.md)
QUIZ Sprint 1 (13–20 Oct 2026)	Containerise, automate and monitor QuizKaro	QUIZ-1 to QUIZ-6, six tasks

Table 2.3: Sprint goals and scope

Member	Roll No.	Responsibility	Issues
Ryaan Devnani	2401023	Application development, Dockerfile, Jenkins	QUIZ-1, QUIZ-5

Member	Roll No.	Responsibility	Issues
Preet Dayaramani	2401021	CI and deployment automation, Netlify	QUIZ-4, QUIZ-6
Dinky Aswani	2401006	Docker Compose, database, monitoring	QUIZ-2, QUIZ-3
Total		3 members	6 tasks

Table 2.4: Distribution of work across the team

2.7 Definition of Done

An issue is moved to Done only when all of the following hold. The pull-request template in the repository carries the same checklist.

1. The branch is named QUIZ-<n>-<slug> and the commit message begins with the issue key.
2. The change reached main through a pull request whose checks, including the Netlify deploy preview, passed.
3. The pull request was reviewed by a teammate and the automated tests pass (npm test), with the page checked in light and dark mode at mobile width.
4. Merging the pull request moved the Jira item to Done through automation, and the behaviour was verified in the running application.

The third and fourth conditions tie the plan to the pipeline: an issue cannot be closed while its pull request is unreviewed or its checks are failing.

3. System Design

3.1 Architecture

QuizKaro is a TypeScript single-page application built with Vite into static files (Figure 3.1). On Netlify the browser downloads these files and keeps all data in localStorage. In the container stack, an nginx container serves the same files on port 8080 and an Express API on port 3000 stores quiz attempts in MySQL, which gives a leaderboard shared by everyone. The front end tries the API first with a short timeout and silently falls back to local data, so the public demonstration never depends on the back end.

Prometheus and Grafana run alongside the application in the same Compose network. Prometheus scrapes the API's /metrics endpoint, Grafana queries Prometheus to draw the dashboard, and phpMyAdmin provides a web screen on the database. Outside the stack, GitHub triggers Netlify and the checks on every push, and Jenkins checks out the repository and runs its pipeline.

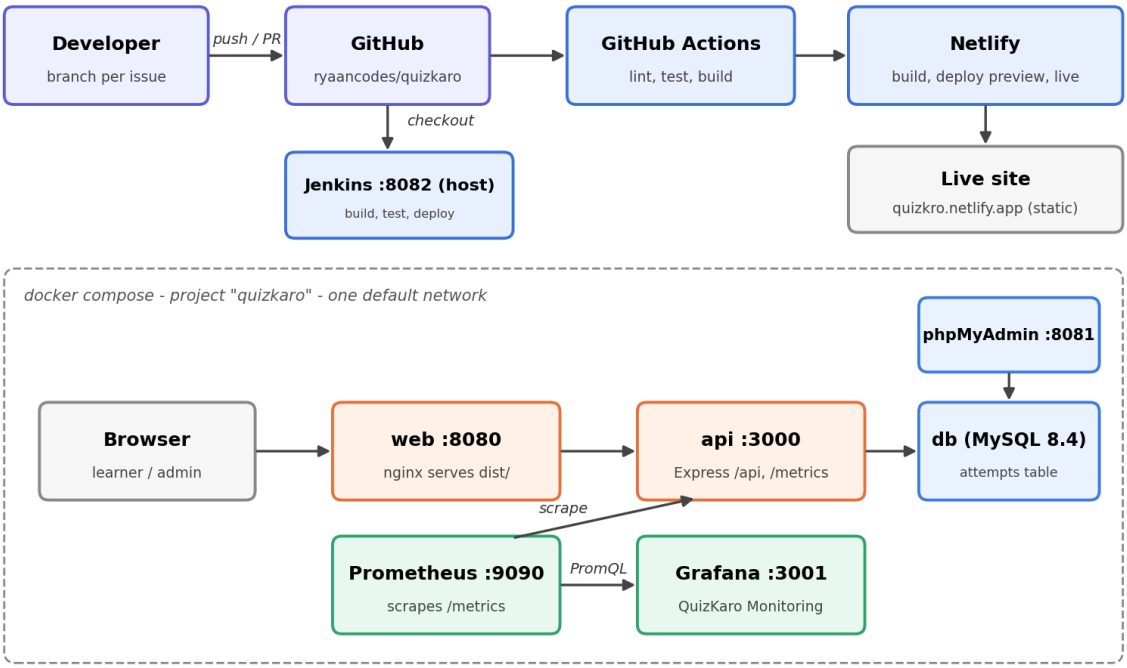


Figure 3.1: System and delivery architecture

3.2 Technology Stack

Layer	Technology	Reason for choice
Build tool	Vite 5	Fast dev server; bundles TypeScript and CSS into small static files for Netlify
Language	TypeScript (strict mode)	Types catch mistakes before the code runs and make data shapes easy to explain
Styling	CSS variables; Tailwind and PostCSS configured	Variables give instant light and dark themes
Reports	jsPDF, SheetJS (xlsx), hand-written CSV	Files are created inside the browser, so no server is needed
Testing	Vitest, jsdom	Fast unit tests plus a simulated full run of the application
Code quality	ESLint (typescript-eslint), Prettier	Consistent style and early bug warnings

Layer	Technology	Reason for choice
Web server	nginx (alpine)	Serves the compiled static files inside the container
API	Node.js, Express 5, mysql2	Small API for the shared leaderboard; simple to explain
Database	MySQL 8.4, phpMyAdmin 5	Relational storage for attempts and a screen to inspect it
Containers	Docker, Docker Compose	Same environment on every machine; one command starts all services
CI/CD	GitHub Actions, Jenkins, Netlify	Automatic checks, previews and deployment
Monitoring	Prometheus, Grafana	Time-series metrics, an alert on API down, dashboards
Planning	Jira (Scrum), GitHub	Sprint board, automation rules, branches and pull requests

Table 3.1: Technology stack

3.3 Data Design

In standalone mode the browser stores JSON records in localStorage under keys beginning qz_. The 28 built-in quizzes live in the code; only custom quizzes are saved. In full-stack mode one MySQL table, attempts, stores each completed attempt for the shared leaderboard (Figure 3.2). An index on quiz_id and points supports the top-five query.

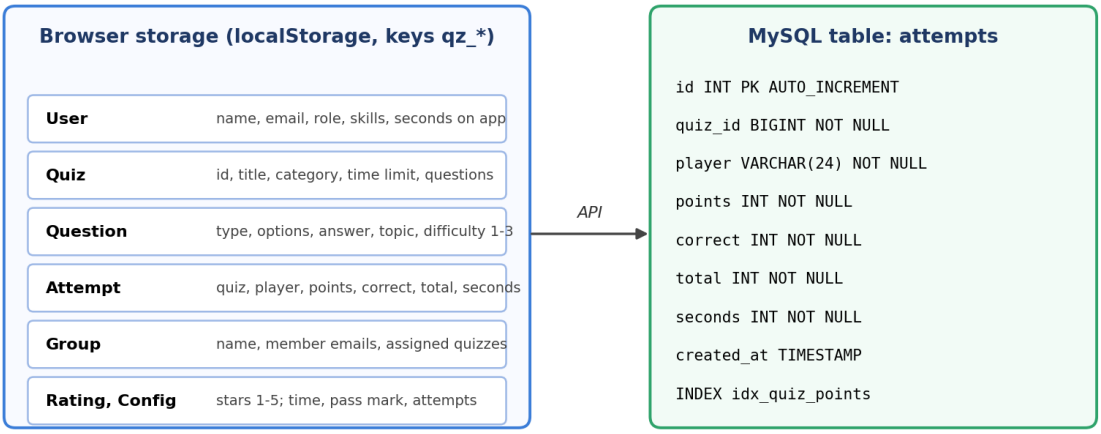


Figure 3.2: Browser records and the MySQL attempts table

3.4 Adaptive Difficulty and Scoring

Every question has a difficulty of 1 (easy), 2 (medium) or 3 (hard), and each quiz has two questions at every level. The quiz starts at level 1 and a small rule-based engine runs after every answer (Figure 3.3). The next question is the unused question whose difficulty is closest to the current level, with ties broken randomly. The engine is deterministic rather than machine learning, which makes it easy to test and to explain; the administrator can switch it off, in which case questions come from a fixed middle level.

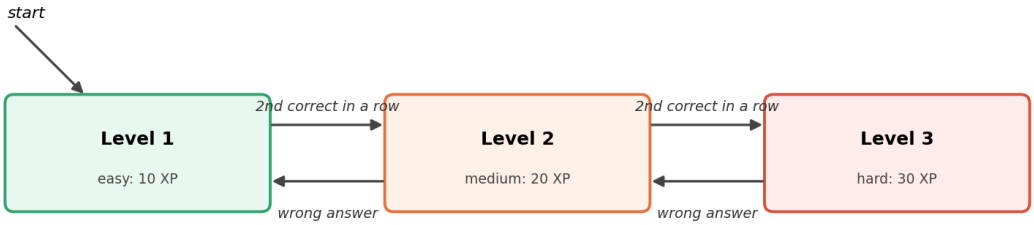


Figure 3.3: Adaptive difficulty rules

A correct answer earns 10, 20 or 30 XP by difficulty, so a perfect quiz of six questions is worth 120 XP. The learner's overall level is level = floor(√(XP ÷ 40)) + 1, which gives the thresholds in Table 3.2.

Level	1	2	3	4	5
XP needed	0	40	160	360	640

Table 3.2: XP needed for each level

3.5 API Design

Method	Endpoint	Purpose
POST	/api/attempts	Validate and store one attempt in MySQL
GET	/api/leaderboard/:quiz	Top five attempts for a quiz
GET	/api/health	Check the database connection
GET	/metrics	Metrics in Prometheus text format: requests and attempts saved

Table 3.3: REST API

3.6 Security Design

- All user-typed text is HTML-escaped before it is shown, which blocks script injection.
- The API validates every attempt before storing it and uses parameterised SQL, so input cannot change the query.
- Database credentials are supplied to the containers as environment variables rather than written into the images, and the MySQL port is not published to the host.
- The production site is static files only, so there is no server-side code on Netlify to attack.
- Honest limit: passwords are stored in plain text in the browser and the role check is made in the browser. This is acceptable for a demonstration only; Section 7.4 describes the production fix.

4. Application Implementation

The code is organised so that logic and screens are separate. The lib folder holds pure functions with no screen code (adaptive rules, grading, statistics, reports, sharing, API client), which can be unit-tested in isolation. The views folder only draws screens and calls that logic. State is held in one store that saves to localStorage, and a single click listener reads data-a and data-v attributes, which keeps the views simple. The look is a warm “poster” design in cream, ink, marigold, peacock teal and vermilion with light and dark themes and a phone-friendly layout.

4.1 Learner Experience

After signing in (Figure 4.1), the learner sees the home page with only two rows of quizzes and a “See all” button, the recommended next quiz and quizzes picked for the skills chosen in Settings (17 skills are available). A quiz shows one question per screen with a segmented progress bar that turns green or red for each answer, a countdown timer and keyboard shortcuts (keys 1 to 4 to answer, Enter for next) (Figure 4.2). The result screen (Figure 4.3) shows the score ring, XP, level, topic bars, pass or fail, the full answer review, the leaderboard and a star rating.

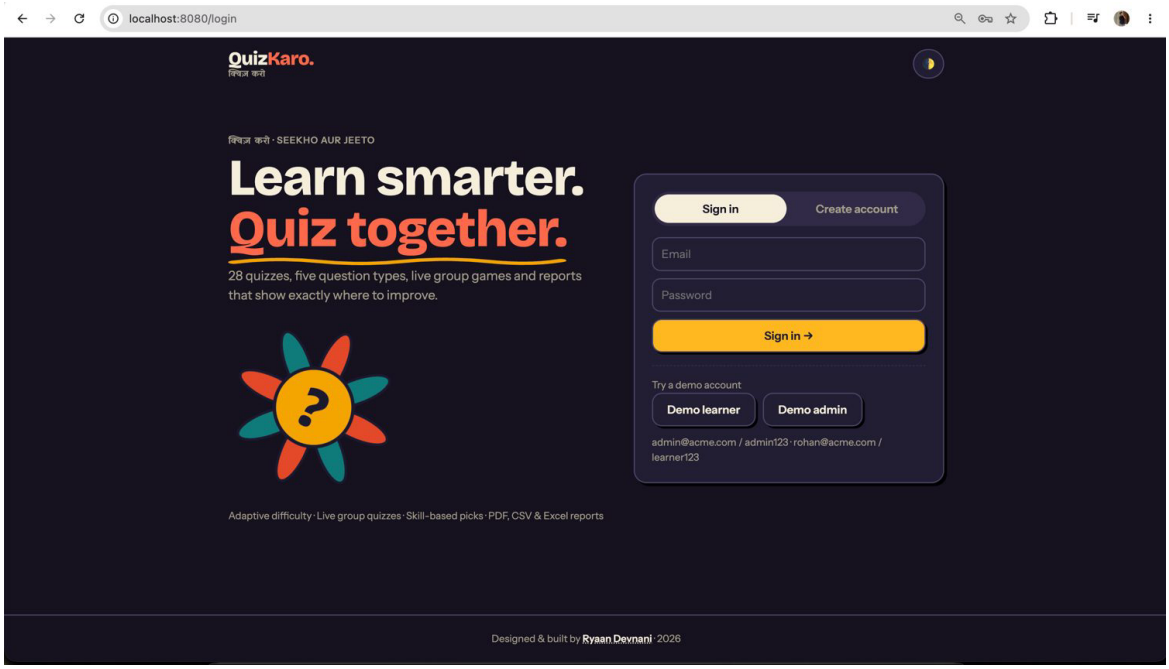


Figure 4.1: Login page with demo-account shortcuts (running in the Docker container on port 8080)

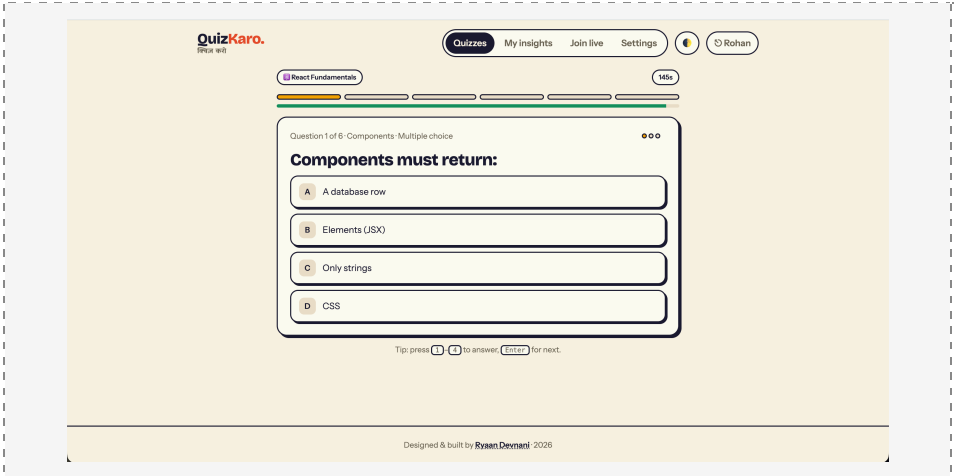


Figure 4.2: Quiz screen with timer and instant feedback

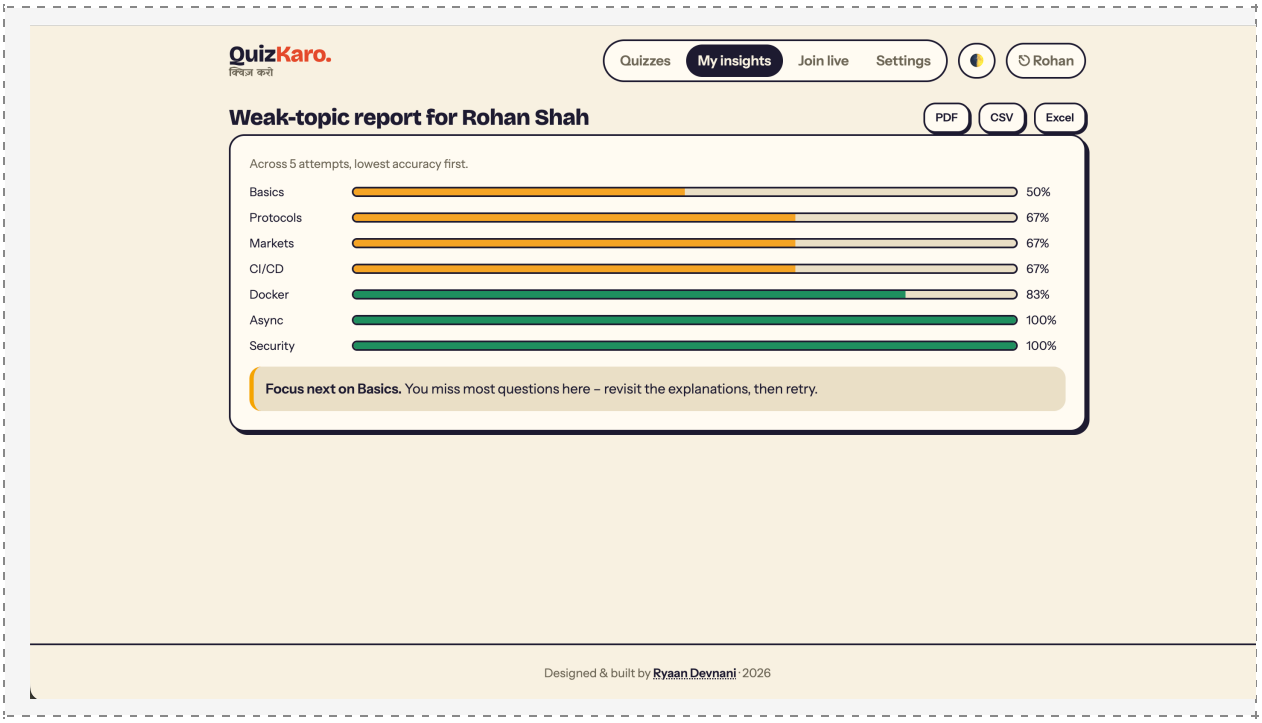


Figure 4.3: Result screen with leaderboard and rating

4.2 Question Types and Grading

Type	What the learner does	How it is graded
Multiple choice	Taps one of four shuffled options	Chosen option equals the correct option
Match the following	Picks the right partner for each item	Every pair must be right
Fill in the blanks	Types a word into each gap in the sentence	Ignores capitals and extra spaces; alternatives separated by |
Drag to order	Drags items into sequence (arrow buttons on touch screens)	The whole sequence must match exactly
Free text	Writes a short answer with a live word counter	Keyword check: at least half of the teacher's key ideas must appear

Table 4.1: Question types and grading rules

Any question can also carry an image, given as a link or an uploaded file. Free-text grading is a transparent keyword rule that the administrator controls; no AI service is used.

4.3 Administrator Tools

The administrator dashboard (Figure 4.4) shows every learner's statistics, hours on the app, accuracy and the top-rated quizzes. Further tabs let the administrator create quizzes with all question types and a preview, form groups by importing learners, assign quizzes, and change the Configuration (seconds per question, maximum duration, word limit, pass mark, attempts allowed, shuffle, adaptive mode and whether explanations and leaderboards are shown). A Learner view toggle lets the administrator see exactly what a learner sees.

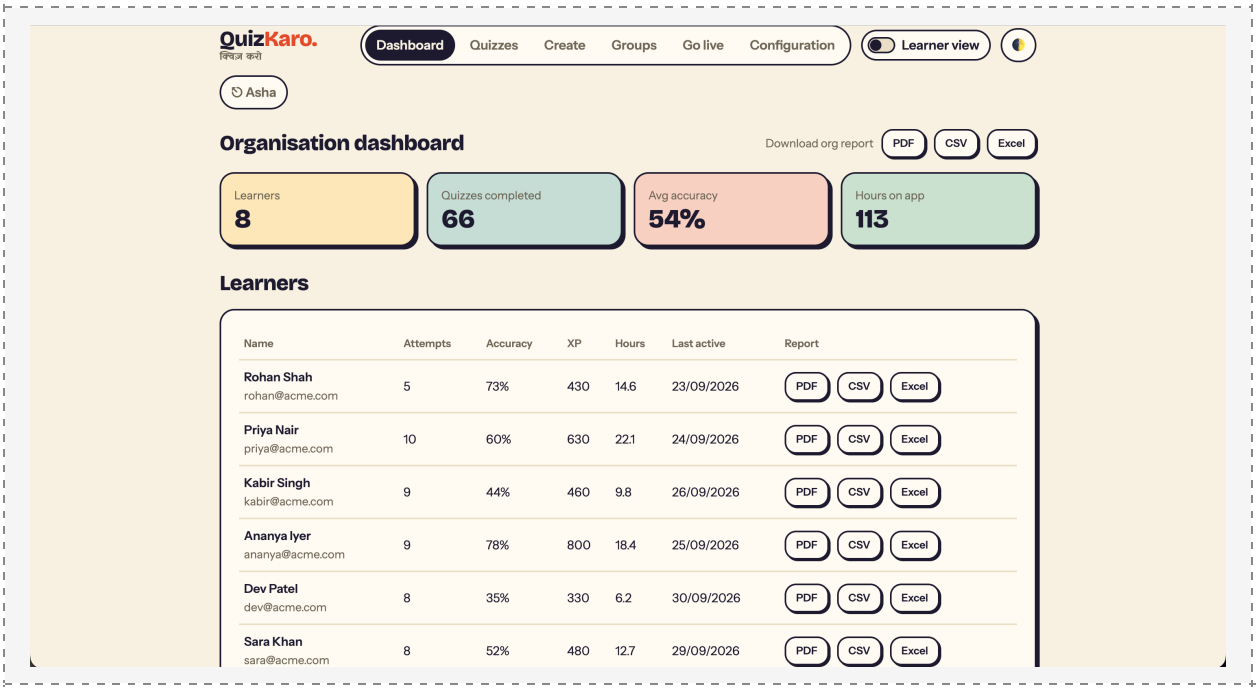


Figure 4.4: Admin dashboard

4.4 Reports

One function builds a report object (summary numbers, two charts, notes and data tables) and three exporters reuse it, so the formats always agree. Everything is generated in the browser with a download timestamp.

Format	Library	Content
PDF	jsPDF	Header band, timestamp, four headline numbers including hours on the app, two bar charts, written insights and a recent-attempts table
Excel	SheetJS	Learner report: Summary, By quiz, Topics, Attempts. Organisation report: Summary, Learners, Weak topics, Attempts
CSV	hand-written	The same tables as text with a byte-order mark so Excel reads it correctly

Table 4.2: Report formats

4.5 Live Group Quiz

The host creates a four-letter room and players join with the code. They exchange small messages over a browser BroadcastChannel: join (player enters), q (host shows the same question with an end time), ans (player locks in an answer), rev (host reveals the answer, explanation and leaderboard) and end (final leaderboard). Points are 100 for a correct answer plus up to 50 for speed. Because BroadcastChannel only links tabs of the same browser, the live quiz works across tabs; across phones it would need a WebSocket server.



Figure 4.5: Live group quiz

4.6 Instrumentation

The Express API exposes /metrics in Prometheus text format with a request counter and a counter of attempts saved, and /api/health, which checks the database connection. These endpoints are what Prometheus scrapes and what the container health checks use. The default up series that Prometheus records for each scrape target is used for the availability panel in Grafana (Section 5.6).

5. DevOps Implementation

This chapter describes how each lab experiment was applied. Figure 5.1 summarises the delivery pipeline that ties them together: a change starts on a branch named after its Jira issue, is checked, previewed and reviewed through a pull request, reaches main by merge, and is then deployed, containerised and monitored.

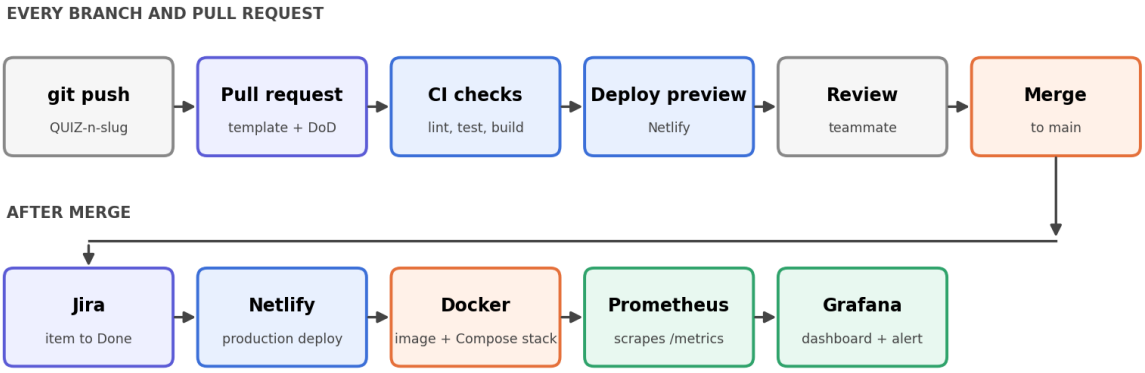


Figure 5.1: Delivery pipeline from push to monitoring

5.1 Version Control with Git and GitHub (Experiment 1)

The repository ryaancodes/quizkaro is hosted on GitHub. The branching rule is one branch per Jira issue, named QUIZ-<n>-<slug>, with every commit message and pull-request title beginning with the issue key. Changes reach main only through pull requests. Pull requests use a template with three headings: What changed, How to test (npm test passes; checked in light and dark mode on mobile width) and Definition of done (reviewed by a teammate; CI is green).

The first issue, QUIZ-1, added package-lock.json so that Docker builds are reproducible. It was developed on its own branch and pushed:

```
$ git checkout -b QUIZ-1-dockerise
$ git add package-lock.json
$ git commit -m "QUIZ-1: add package-lock.json for reproducible Docker builds"
$ git push -u origin QUIZ-1-dockerise
$ git log --oneline --graph
* 5c43982 (HEAD -> QUIZ-1-dockerise, origin/QUIZ-1-dockerise) QUIZ-1: add package-lock.json
...
* 83f6af6 (origin/main, origin/HEAD, main) Add Quizkaro project
* c369c7e first commit
```

Pull request #1 (QUIZ-1: add package-lock.json for reproducible Docker builds) merged one commit from QUIZ-1-dockerise into main, adding 4,981 lines (Figure 5.2). Netlify posted a deploy preview on the pull request, and the checks passed with no conflicts (Figure 5.3); the pull request was then merged and the branch could be deleted (Figure 5.4). The same sequence of terminal commands was then repeated for QUIZ-2-docker-compose (Figure 5.5).

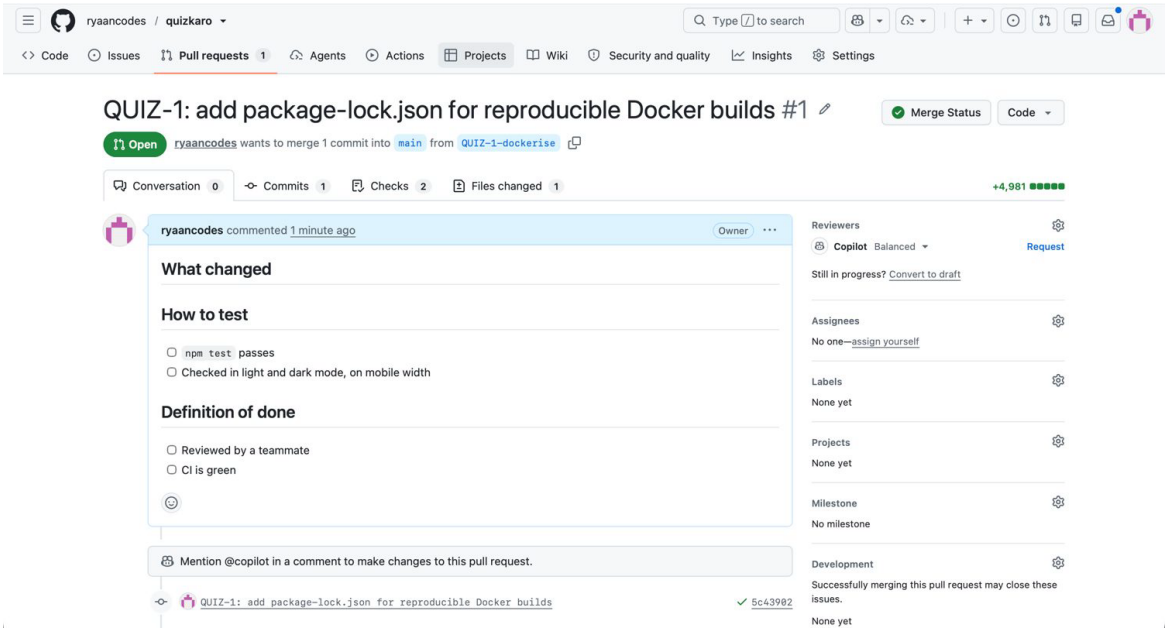


Figure 5.2: Pull request #1 for QUIZ-1 with the template checklist

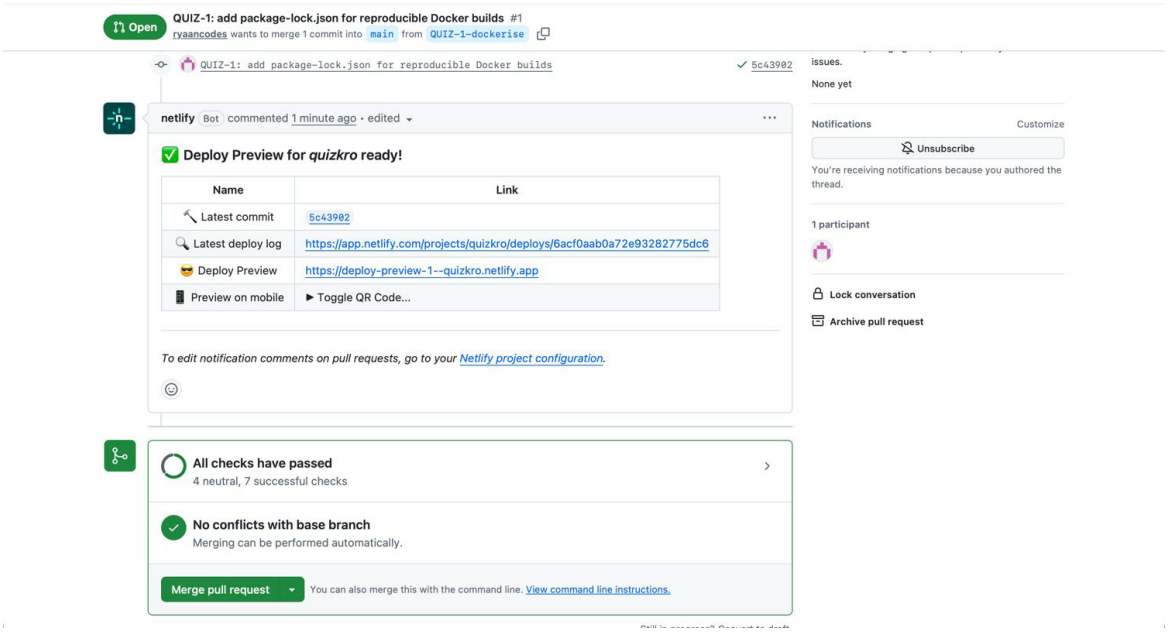


Figure 5.3: Netlify deploy preview and passing checks on the pull request

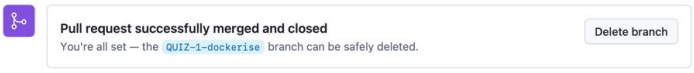
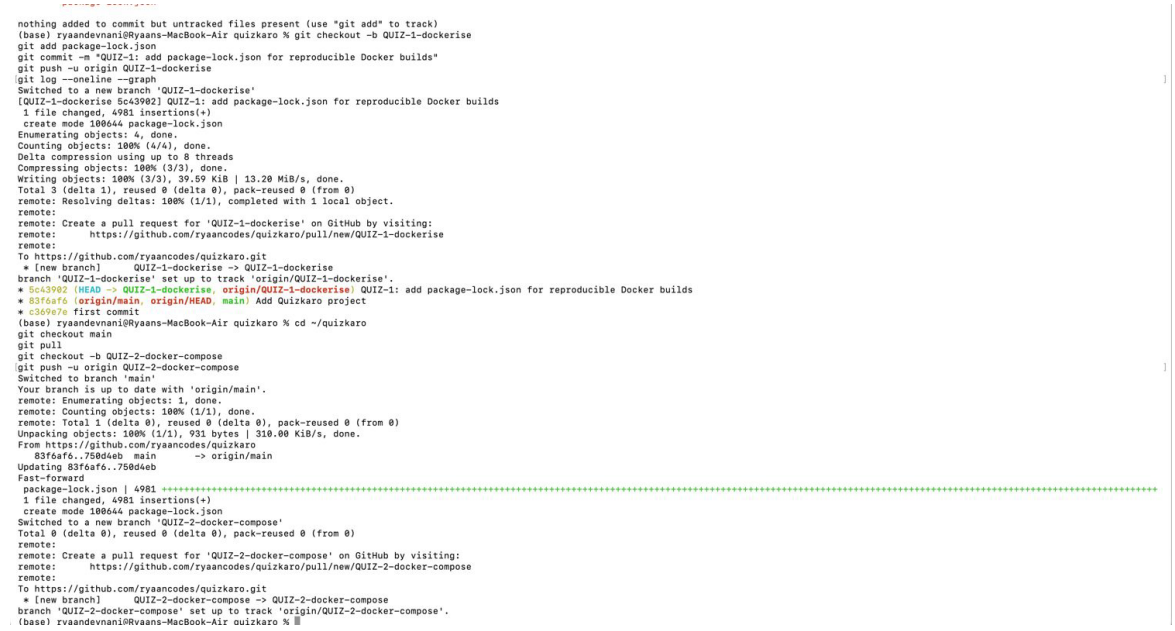


Figure 5.4: Pull request merged and closed


nothing added to commit but untracked files present (use "git add" to track)
(base) ryaandevnani@Ryaans-MacBook-Air quizkaro % git checkout -b QUIZ-1-dockerise
git add package-lock.json
git commit -m "QUIZ-1: add package-lock.json for reproducible Docker builds"
git push -u origin QUIZ-1-dockerise
git log --oneline --graph
Switched to a new branch 'QUIZ-1-dockerise'
[QUIZ-1-dockerise 5c43902] QUIZ-1: add package-lock.json for reproducible Docker builds
1 file changed, 4981 insertions(+)
create mode 100644 package-lock.json
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 39.59 KiB | 13.20 MiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'QUIZ-1-dockerise' on GitHub by visiting:
remote: https://github.com/ryaancodes/quizkaro/pull/new/QUIZ-1-dockerise
remote:
To https://github.com/ryaancodes/quizkaro.git
* [new branch] QUIZ-1-dockerise -> QUIZ-1-dockerise
branch 'QUIZ-1-dockerise' set up to track 'origin/QUIZ-1-dockerise'.
* 5c43902 (HEAD -> QUIZ-1-dockerise, origin/QUIZ-1-dockerise) QUIZ-1: add package-lock.json for reproducible Docker builds
* 83f6af6 (origin/main, origin/HEAD, main) Add Quizkaro project
* c369e7e first commit
(base) ryaandevnani@Ryaans-MacBook-Air quizkaro % cd ~/quizkaro
git checkout main
git pull
git checkout -b QUIZ-2-docker-compose
git push -u origin QUIZ-2-docker-compose
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
remote: Enumerating objects: 1, done.
remote: Counting objects: 100% (1/1), done.
remote: Total 1 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (1/1), 931 bytes | 310.00 KiB/s, done.
From https://github.com/ryaancodes/quizkaro
83f6af6..750d4eb main -> origin/main
Updating 83f6af6..750d4eb
Fast-forward
package-lock.json | 4981 ++++++

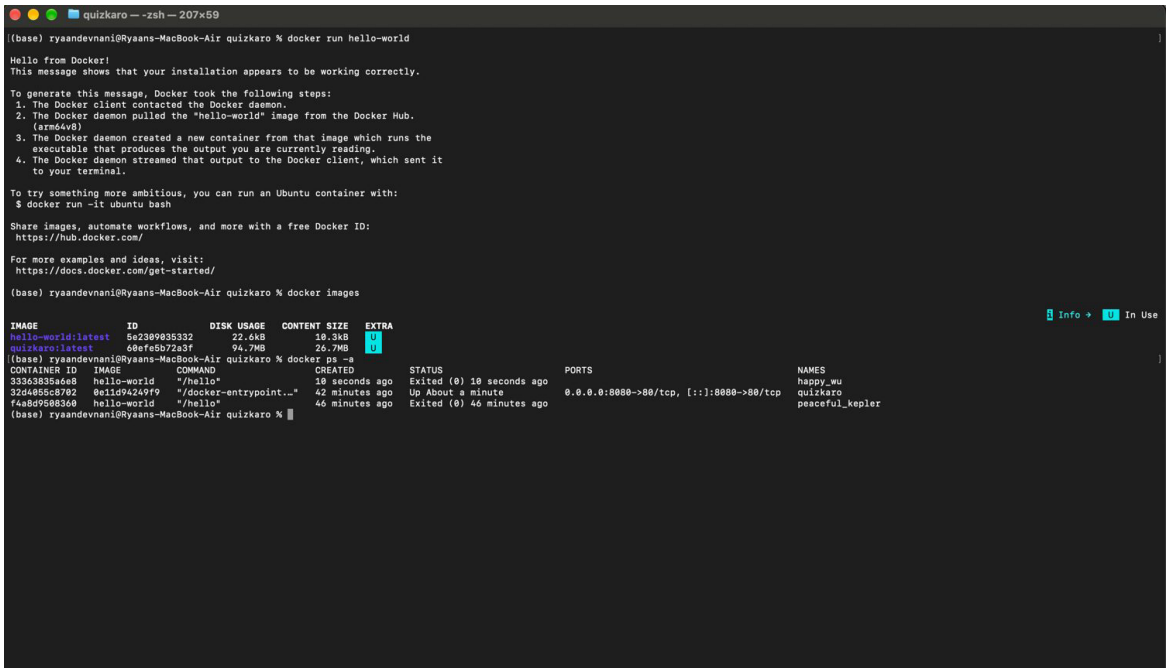


Figure 5.6: Hello World test, docker images and docker ps -a

The image was then built with docker build -t quizkaro . and the build finished successfully (Figure 5.7). The container was run with its port 80 mapped to port 8080 on the host so the site is reachable in the browser (Figure 4.1). The commands used to operate it are listed in Table 5.1 and their output in Figure 5.8.

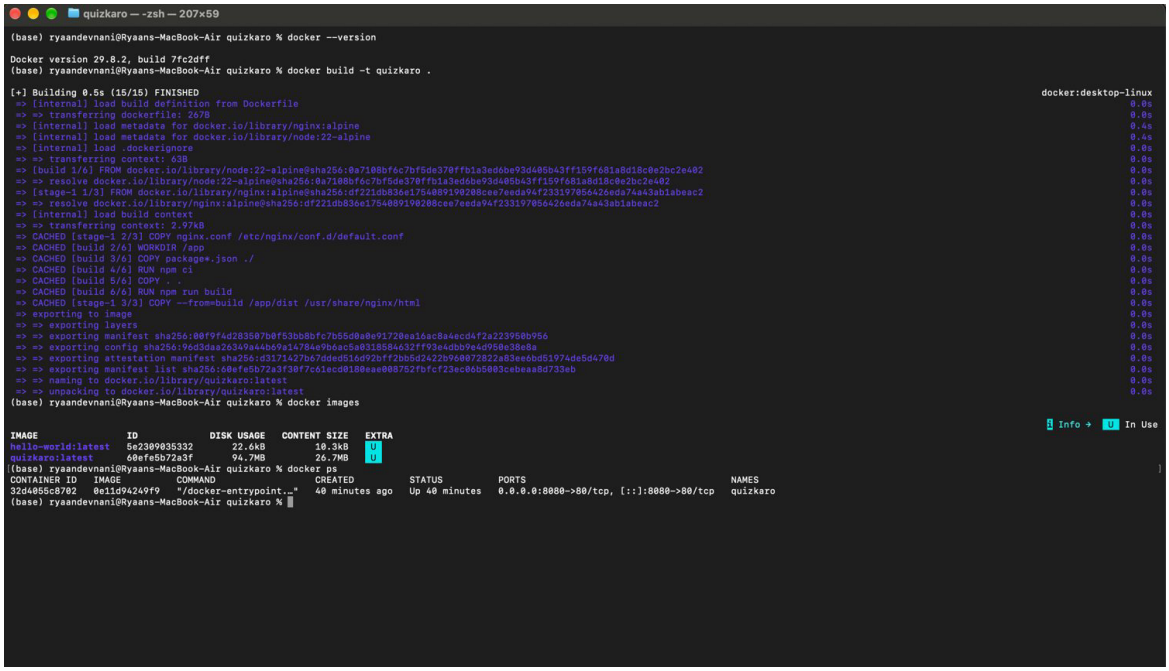


Figure 5.7: docker --version, docker build and the resulting image

Command	Purpose
docker build -t quizkaro .	Build the image from the Dockerfile
docker run -d --name quizkaro -p 8080:80 quizkaro	Run the container and map host port 8080 to container port 80
docker logs quizkaro	Show nginx start-up and the access log of browser requests
docker exec quizkaro ls /usr/share/nginx/html	Confirm that only the compiled site is inside the image

Command	Purpose
docker stats --no-stream	Read CPU, memory and process count
docker stop quizkaro / docker start quizkaro	Stop and restart the container

Table 5.1: Docker commands used

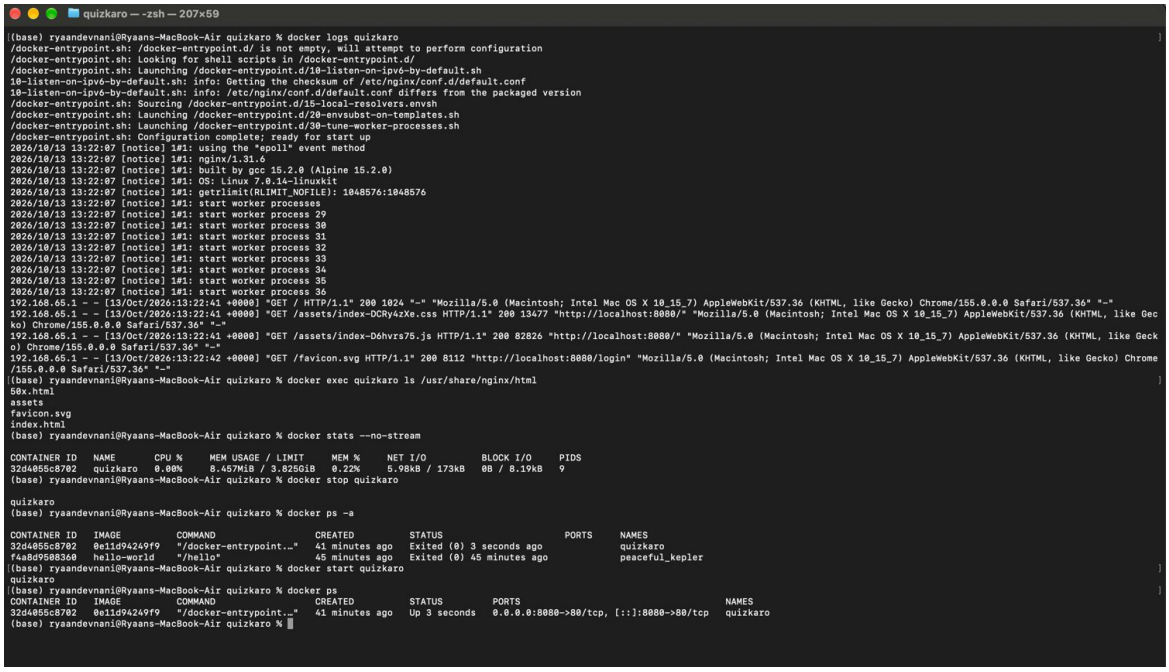


Figure 5.8: Container logs, contents of the web root, resource usage, stop and start



Figure 5.9: git diff of .dockerignore, Dockerfile and nginx.conf

5.3 Multi-Container Orchestration with Docker Compose (Experiment 3)

A single docker-compose.yml defines six services under the project name quizkaro (Table 5.2). The database is started first and the stack output reports it Healthy before the other containers start; the MySQL data and the Grafana data live in named volumes (quizkaro_db_data and quizkaro_grafana_data), so they survive restarts. All services join the network quizkaro_default and reach each other by service name. Images that were not built locally (Grafana, phpMyAdmin, Prometheus and MySQL) were pulled once, which took between five and six and a half minutes each, and the two application images were built in about 57 seconds.

Service	Image	Host port	Notes
web	quizkaro-web (built)	8080	nginx serving the compiled site
api	quizkaro-api (built)	3000	Express API: /api/* and /metrics
db	mysql:8.4	not published	Health check; volume quizkaro_db_data; database quizly
phpmyadmin	phpmyadmin:5	8081	Web screen on the database
prometheus	prom/prometheus	9090	Scrapes the API's /metrics
grafana	grafana/grafana	3001	Dashboards; volume quizkaro_grafana_data

Table 5.2: Services in the Compose stack

Host port 3001 is used for Grafana because the API already holds 3000, and MySQL is deliberately not published to the host so that it cannot clash with a MySQL server installed on the machine. Twelve seconds after docker compose up, every container was listed as Up, with the database healthy and the web container in its health start-up period (Figure 5.10).



Figure 5.10: docker compose up: images pulled and built, volumes, network and six containers started

To verify the whole chain, a quiz attempt was made in the application and then looked up in phpMyAdmin at port 8081. The server page shows MySQL 8.4 reached over TCP/IP as user quizly (Figure 5.11); the quizly database holds one table, attempts (Figure 5.12), and after the quiz it contained a row for player Rohan Shah on quiz 2 with 10 points, 1 correct of 6, taken in 9 seconds (Figure 5.13).

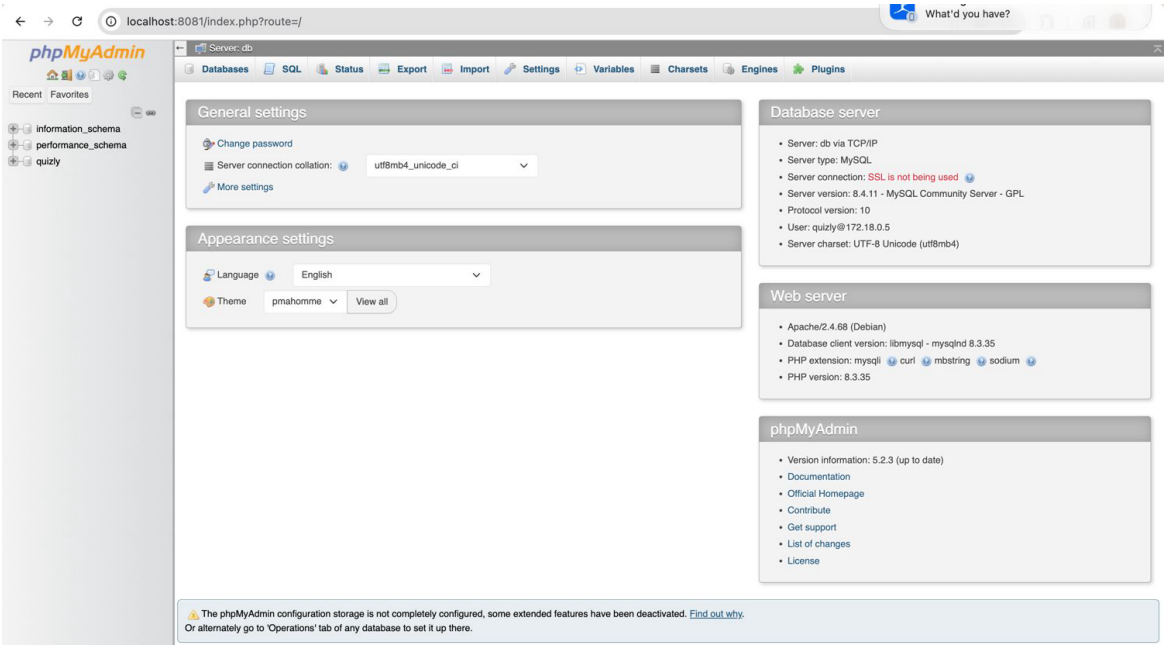


Figure 5.11: phpMyAdmin connected to the db service

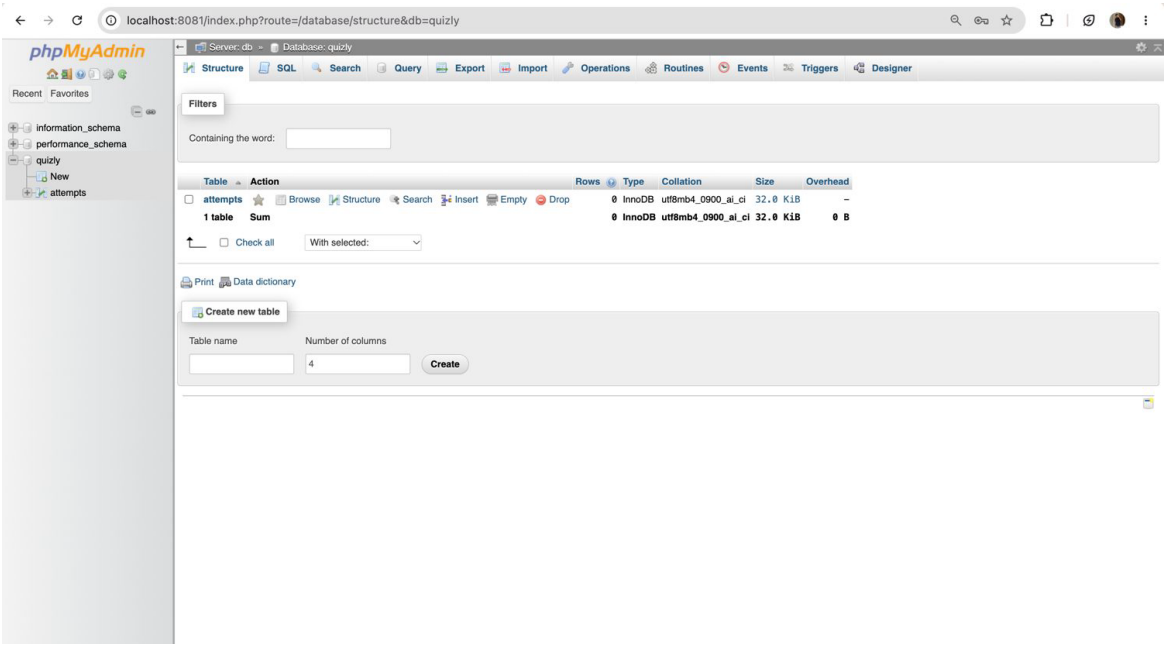


Figure 5.12: The quizly database with the attempts table

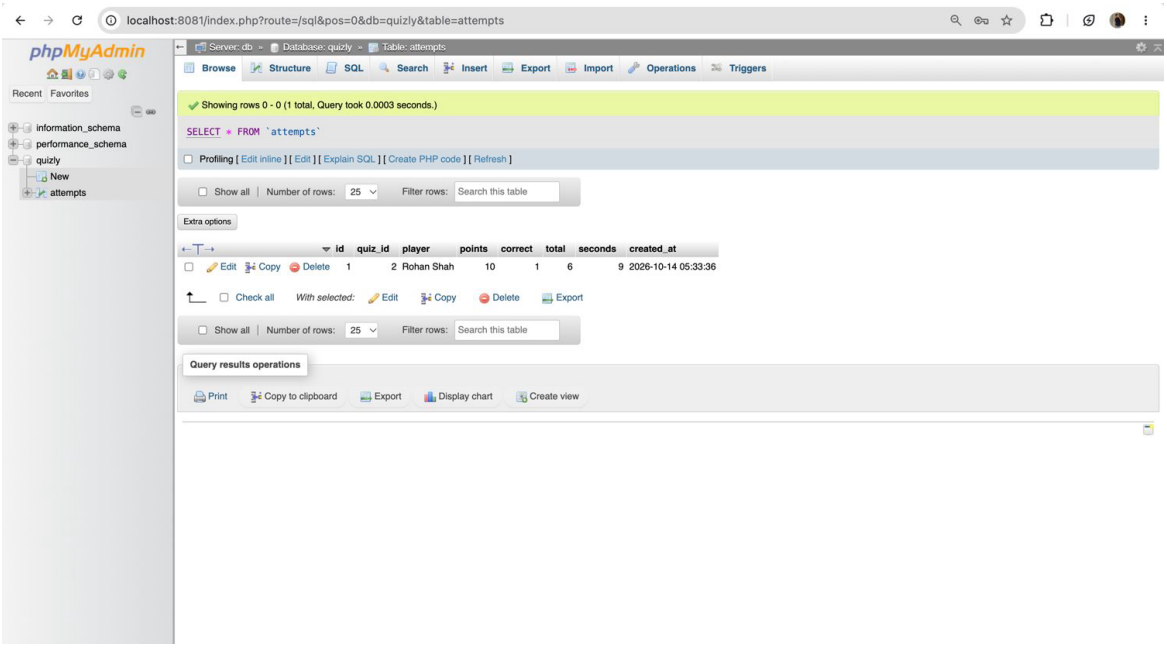


Figure 5.13: A stored attempt in the attempts table

5.4 Continuous Integration with Jenkins (Experiment 4)

Jenkins runs locally from the Jenkins LTS war file on port 8082, because 8080 is used by the web container. A pipeline job named GitHub-Jenkins-Demo is connected to a GitHub repository and checks out its Jenkinsfile on every build. The pipeline is declarative, as in Experiment 4, and is stored in the repository so that it is versioned and reviewed like any other code. Its stages, as shown in the build console, are summarised in Table 5.3.

Stage	What the console shows	Purpose
Checkout	Clones the repository, fetches refs, checks out the commit “Create Jenkinsfile”	Get the exact source that is to be built
Build	“Building the application...”	Compile or package the application
Test	“Testing the application...”	Run the automated tests
Deploy	“Deploying the application...”	Release the build

Table 5.3: Pipeline stages

The job page lists the successful builds: #1 and #2 on 17 August 2026 and #4 on 14 October 2026, with the last build also being the last stable and last successful one (Figure 5.14). The console of build #4 ends with “End of Pipeline” and no errors (Figure 5.15). The same three-stage structure is what the QuizKaro repository’s own Jenkinsfile follows (install, lint, test, build, then deploy to Netlify on main with stored credentials), as described in Section 7.4.

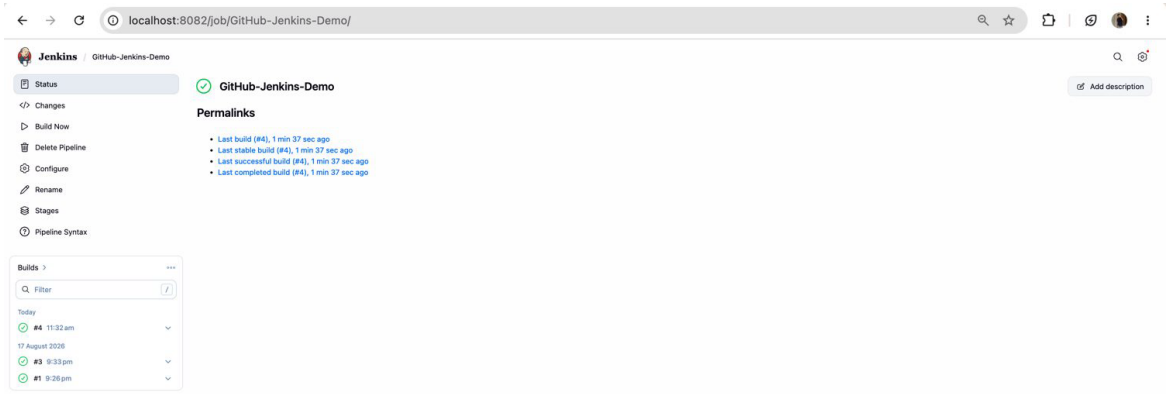


Figure 5.14: Jenkins job GitHub-Jenkins-Demo with successful builds

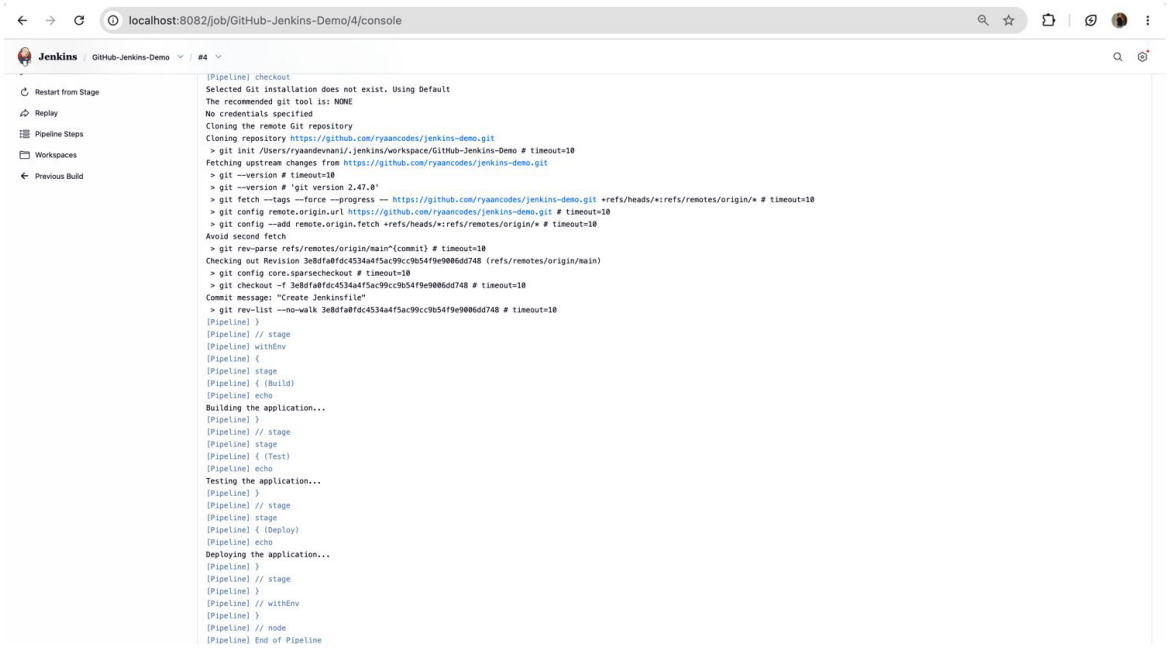


Figure 5.15: Console output of Jenkins build #4

5.5 Automated Deployment (Experiment 5)

Automated deployment of QuizKaro is performed by Netlify. The repository contains netlify.toml with the build command npm run build, the publish folder dist and Node 20. Every push to main triggers a new build and the live site at quizkro.netlify.app updates in about a minute without any manual upload. Every pull request additionally receives a deploy preview URL, which was posted on pull request #1 as https://deploy-preview-1--quizkaro.netlify.app (Figure 5.3). The Jenkins Deploy stage in Table 5.3 is the equivalent hook for a pipeline-driven release.

Channel	Trigger	Result
Netlify production	Push or merge to main	npm run build, publish dist/, live site updated
Netlify deploy preview	Pull request opened or updated	Preview URL commented on the pull request

Channel	Trigger	Result
Jenkins Deploy stage	Jenkins build of the Jenkinsfile	Deploy step of the pipeline (Table 5.3)
Docker Compose	docker compose up --build	Full stack with API, database and monitoring on one host

Table 5.4: Deployment channels

5.6 Monitoring (Experiment 7)

Experiment 7 contrasted state-based monitoring with the time-series model of Prometheus and Grafana, which records how a value has behaved. QuizKaro uses the time-series model. Prometheus scrapes the API's /metrics endpoint every 15 seconds, and the metric up records for each target whether the scrape succeeded. An alert rule fires if the API has been down for one minute.

Metric or alert	Type	Meaning
Request counter	counter	Requests handled by the API
Attempts-saved counter	counter	Quiz attempts stored in MySQL
up{job="quizkaro-api"}	gauge	1 while the API scrape succeeds, 0 when it fails
API down alert	alert rule	Fires when the API has been down for one minute

Table 5.5: Metrics and alert

Grafana runs at port 3001. Its Prometheus data source was added and the Test button reported "Successfully queried the Prometheus API" (Figure 5.16). A dashboard named QuizKaro Monitoring was then created with a Service up/down panel that plots up for the quizkaro-api job (instance api:3000) and for Prometheus itself over the last six hours (Figure 5.17). Stopping the API with docker compose stop api, as shown in Figure 5.10, makes the API series fall to 0, which is the condition the alert rule watches.

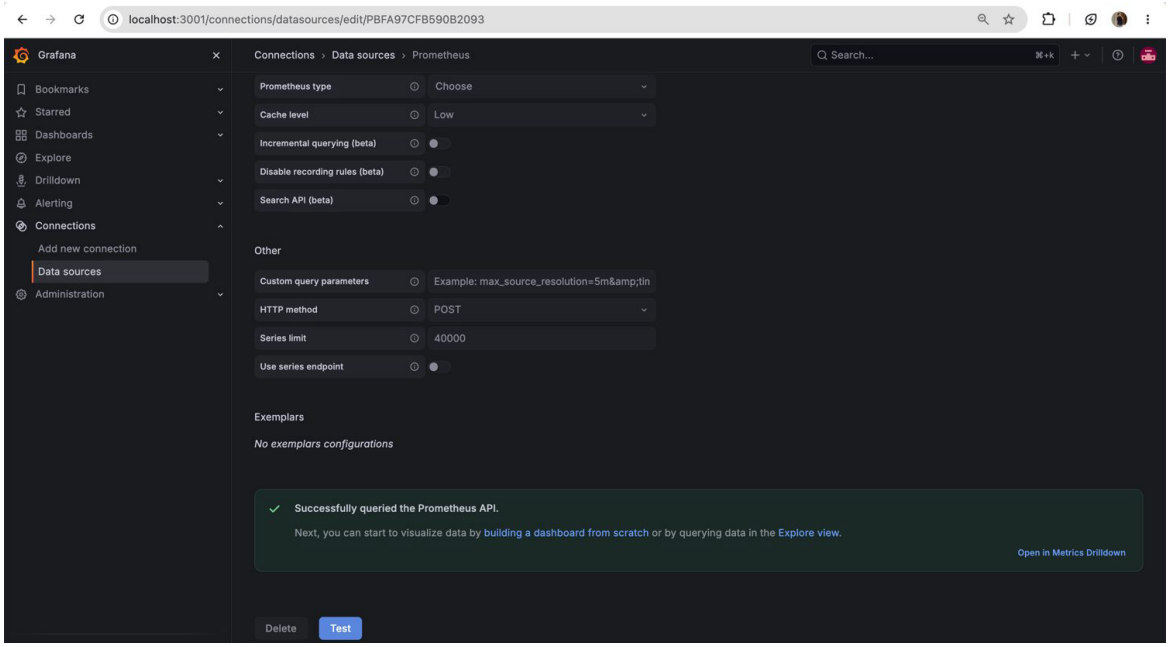


Figure 5.16: Grafana data source for Prometheus, test successful

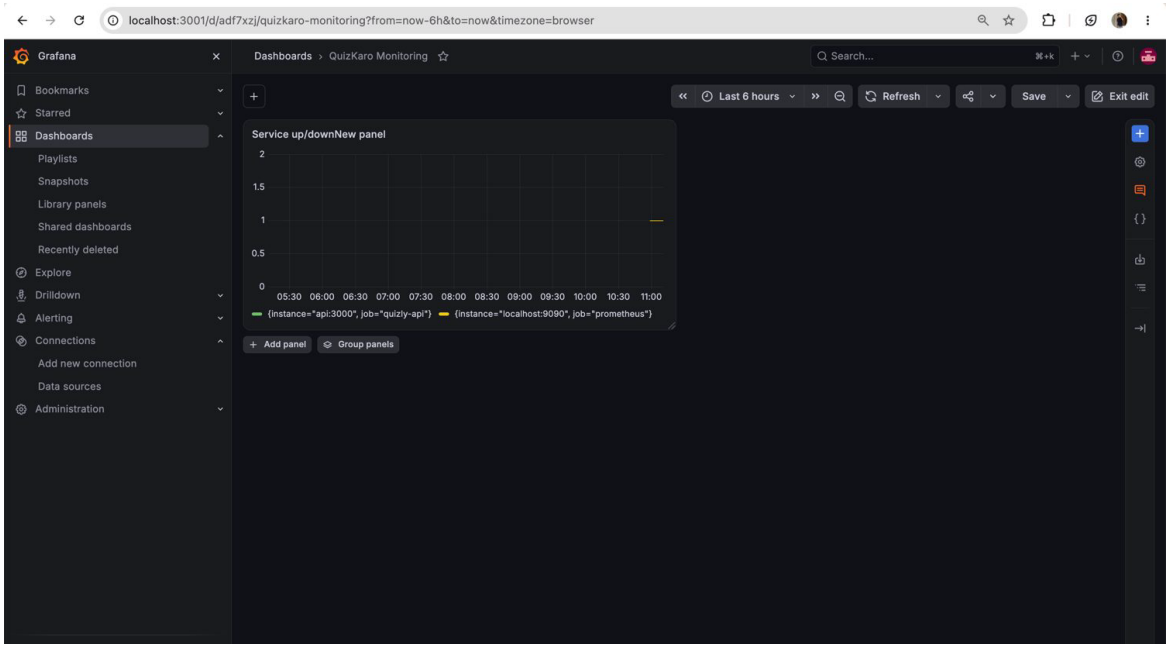


Figure 5.17: QuizKaro Monitoring dashboard with the Service up/down panel

6. Testing

6.1 Automated Tests

Vitest with jsdom runs 20 automated tests. The core logic is kept in pure functions so that it can be tested without a browser, and a simulated end-to-end test plays a complete quiz through the interface.

Test file	Tests	What it proves
adaptive.test.ts	6	Level rises after two correct and falls after a miss; question picking; XP weighting; leaderboard order; topic counts
grade.test.ts	6	Every question type is graded correctly, including alternatives and word counting
catalog.test.ts	4	At least 28 quizzes; each has all three difficulty levels; every question is well-formed; all five types present; skills point at real quizzes; level formula
smoke.test.ts	4	Simulated users: login, home page, a full mixed-type quiz to the result and rating, learner pages, admin pages and learner-view toggle
Total	20	

Table 6.1: Automated test suites

6.2 Quality Checks

Check	Tool	Result
Type-check	TypeScript, strict mode	Passed
Unit and smoke tests	Vitest (20 tests)	Passed
Lint	ESLint	No problems reported
Production build	Vite with Tailwind and PostCSS	Succeeded

Table 6.2: Checks run on the final code

6.3 Container and Deployment Verification

Beyond unit tests, the deployment itself was verified. The container was started and its nginx access log recorded the browser's requests for the page and its assets with HTTP 200; the web root listing contained only the compiled site; docker stats gave the readings in Table 6.3; and after docker stop and docker start the container came back and served the site again. For the full stack, the quiz attempt that appeared in phpMyAdmin proves that browser, API and database work together.

Measure	Value
Web image (docker images)	94.7 MB disk usage, 26.7 MB content size
First build of the web image	8.5 s with cached layers (15 steps)
Build of API and web images in Compose	about 57 s
Stack start-up	all six containers Up 12 s after docker compose up; database healthy after 11.1 s
Idle web container	CPU 0.00 %, memory 8.47 MiB, 9 processes
Stored attempt	1 row in quizly.attempts (10 points, 1 of 6 correct, 9 s)

Table 6.3: Container and stack measurements

7. Results and Discussion

7.1 Outcomes against Objectives

Objective	Outcome
Quizzes, five question types, feedback, leaderboard, rating	Delivered: 28 quizzes, 168 questions, five types
Adaptive difficulty, XP, levels, recommendations	Delivered: rule-based engine, 6 + 6 + 4 + 4 tests
Admin dashboard, groups, configuration, live quiz, reports	Delivered: PDF, CSV and Excel for learner and organisation
Scrum planning with a Definition of Done	QUIZ Sprint 1 with six issues; QZ-1 to QZ-13 for application stories
Branch per issue, pull-request merges	QUIZ-1 branch, pull request #1 and merge shown; QUIZ-2 branch created
Containerised stack, one command	Six services start with docker compose up
Automated build, test and deploy	Netlify deploys main and previews pull requests; Jenkins pipeline ran Build, Test, Deploy
Monitoring and alert on API down	Prometheus scraping; Grafana dashboard with Service up/down panel

Table 7.1: Results against objectives

7.2 The Feedback Loop in Practice

The clearest demonstration of the practices working together is the life of issue QUIZ-1. It was planned in the sprint backlog; when its branch QUIZ-1-dockerise was pushed, the Jira automation rule moved it to In Progress with no manual step (Figure 2.5). Opening pull request #1 moved it to In Review, Netlify built a deploy preview and the checks passed (Figure 5.3), and merging the pull request moved the issue to Done. The plan, the code, the check and the board therefore stayed in agreement without anyone updating them by hand.

7.3 Challenges and Resolutions

Challenge	Resolution
Docker builds were not reproducible without a lock file	package-lock.json was added in QUIZ-1 (4,981 lines) and the Dockerfile uses npm ci
Build tools would bloat the final image	Multi-stage build: only dist/ and the nginx configuration reach the final image (94.7 MB)
Port clashes: Jenkins default 8080 versus the web container; Grafana versus the API on 3000	Web on 8080, Jenkins on 8082, API on 3000, Grafana on 3001, phpMyAdmin on 8081; MySQL not published
Netlify serves static files only, so the API and database cannot run there	Optional back end: the front end tries /api first and silently falls back to local data
Large images took several minutes to pull	Pulled once; later runs reuse the cache
Board status drifting from the real state of the work	Three Jira automation rules linked to GitHub branch and pull-request events

Table 7.2: Challenges met and how they were resolved

7.4 Limitations

- Data is stored per browser in standalone mode, so an administrator only sees learners who used the same browser. A production system would use the Express and MySQL back end for everyone.
- Passwords are stored as plain text and roles are checked in the browser. A production version needs hashed passwords (bcrypt), sessions or JWT and server-side role checks.
- The live quiz uses BroadcastChannel, which links tabs of one browser only; across devices it needs WebSockets.
- CI/CD Pipeline: The project includes a Jenkinsfile defining Build, Test, and Deploy stages. However, the pipeline has not been executed on a Jenkins server, so its functionality remains unverified.
- At the time the screenshots were taken, the board showed QUIZ-1 and QUIZ-2 in progress and QUIZ-3 to QUIZ-6 still in To Do; the monitoring dashboard has the availability panel only.
- Nagios was not used; monitoring is limited to Prometheus and Grafana, and alerts are evaluated but not routed to e-mail or chat because no Alertmanager is deployed.
- Free-text grading is keyword based and no AI marking or AI question generation is included.

8. Conclusion and Future Scope

8.1 Conclusion

QuizKaro meets its functional goal: a learner can take an adaptive, timed quiz, understand each answer, see weak topics and track XP and level, while an administrator creates quizzes, groups learners, hosts live quizzes and downloads reports. More importantly for this course, the project shows the lab experiments operating on one product rather than as separate exercises.

Planning the work as a Jira sprint produced issue keys that appeared in every branch, commit and pull request, and the automation rules kept the board in step with GitHub. Git and pull requests with a Definition of Done made every change reviewable, and Netlify previews let each change be seen before it was merged. Docker and Docker Compose made the environment a versioned artifact, so the full stack of web, API, database, phpMyAdmin, Prometheus and Grafana starts with one command. Jenkins demonstrated a Build, Test and Deploy pipeline, and Prometheus and Grafana made the availability of the API visible over time. The practical lesson repeated across the project is that each practice is most valuable for what it enables in the next one.

8.2 Future Scope

- A secure back end for real multi-user data: hashed passwords, sessions or JWT and server-side authorisation.
- WebSocket-based live quizzes that work across phones, with a server clock for fair timing.
- AI-assisted marking of free-text answers and AI question generation with a teacher review screen.
- More metrics (latency, errors, quizzes per hour), an Alertmanager routing alerts to the team, and Nagios checks for hosts.
- Deployment of the container stack to a cloud host or Kubernetes, and branch protection with required reviews once every member works from their own account.

References

1. Docker Inc., Docker Documentation, https://docs.docker.com/
2. Docker Inc., Docker Compose Documentation, https://docs.docker.com/compose/
3. Jenkins Project, Jenkins User Documentation, https://www.jenkins.io/doc/
4. Prometheus Authors, Prometheus Documentation, https://prometheus.io/docs/
5. Grafana Labs, Grafana Documentation, https://grafana.com/docs/
6. Atlassian, Jira Software Documentation (Scrum and Automation), https://support.atlassian.com/jira-software-cloud/
7. GitHub, GitHub Docs: Pull Requests, https://docs.github.com/en/pull-requests
8. Netlify, Netlify Docs: Deploy Previews, https://docs.netlify.com/
9. Vite, Vite Guide, https://vite.dev/guide/
10. Vitest, Vitest Documentation, https://vitest.dev/
11. Oracle, MySQL 8.4 Reference Manual, https://dev.mysql.com/doc/
12. K. Schwaber and J. Sutherland, The Scrum Guide, https://scrumguides.org/

A. Appendix: Running the Project

Live site: https://quizkro.netlify.app/ Source code: https://github.com/ryaancodes/quizkaro

A.1 Standalone mode

```
git clone https://github.com/ryaancodes/quizkaro.git
cd quizkaro
npm install
npm run dev      # local development server
npm test         # run the 20 tests
npm run lint     # ESLint
npm run build    # type-check and build into dist/
```

A.2 Full stack with Docker Compose

```
docker compose up --build
# app         http://localhost:8080
# API         http://localhost:3000/api/health
# phpMyAdmin  http://localhost:8081
# Prometheus  http://localhost:9090
# Grafana     http://localhost:3001

docker compose stop api     # stop the API to see it go down in Grafana
docker compose down          # stop and remove the containers
```

A.3 Demonstration accounts

Role	Email	Password
Admin	admin@acme.com	admin123
Learner	rohan@acme.com	learner123
Learners	priya, kabir, ananya, dev, sara, vikram, meera @acme.com	learner123

Table A.1: Demonstration accounts